Main application/interface
App.jsx

```
import { useState } from "react"

import { AlertCircle, Cpu, Play, RotateCcw } from "lucide-react"


import { Button } from "@/components/ui/button"

import { Input } from "@/components/ui/input"

import { Label } from "@/components/ui/label"

import {

  Select,

  SelectContent,

  SelectItem,

  SelectTrigger,

  SelectValue,

} from "@/components/ui/select"

import {

  Card,

  CardContent,

  CardDescription,

  CardHeader,

  CardTitle,

} from "@/components/ui/card"

import {

  Table,

  TableBody,

  TableCell,

  TableFooter,

  TableHead,
```

Main application/interface
App.jsx

```
  TableHeader,
  TableRow,
} from "@/components/ui/table"
import { Alert, AlertDescription, AlertTitle } from "@/components/ui/alert"
import { ThemeToggle } from "@/components/theme-toggle"
import { GanttChart } from "@/components/gantt-chart"
import { runSchedule } from "@/lib/scheduling"
import { validateProcesses } from "@/lib/validate"

const ALGORITHMS = {
  fcfs: "First-Come, First-Served (FCFS)",
  sjf: "Shortest Job First (SJF)",
}

function makeRows(count) {
  return Array.from({ length: count }, (_, i) => ({
    id: `P${i + 1}`,
    arrival: "",
    burst: "",
  }))
}

export default function App() {
  const [countInput, setCountInput] = useState("")
  const [rows, setRows] = useState([])
  const [algorithm, setAlgorithm] = useState("fcfs")
```

Main application/interface
App.jsx

```
 const [errors, setErrors] = useState([])

 const [result, setResult] = useState(null)


 function handleSetCount(value) {

  const n = Number(value)

  setCountInput(value)

  setRows(makeRows(n))

  setErrors([])

  setResult(null)

 }


 function updateRow(index, field, value) {

  setRows((prev) =>

   prev.map((row, i) => (i === index ? { ...row, [field]: value } : row))

  )

 }


 function handleCompute() {

  const validationErrors = validateProcesses(rows)

  if (validationErrors.length > 0) {

   setErrors(validationErrors)

   setResult(null)

   return

  }


  const processes = rows.map((r) => ({
```

Main application/interface
App.jsx

```
    id: r.id.trim(),
    arrival: Number(r.arrival),
    burst: Number(r.burst),
  }))

  setErrors([])
  setResult(runSchedule(algorithm, processes))
 }

 function handleReset() {
  setCountInput("")
  setRows([])
  setAlgorithm("fcfs")
  setErrors([])
  setResult(null)
 }

 return (
  <div className="min-h-screen bg-background">
   <header className="mx-auto flex max-w-5xl items-center justify-between px-6 pt-8 pb-6">
    <div className="flex items-center gap-3">
     <div className="flex size-9 items-center justify-center rounded-md bg-primary text-primary-foreground">
      <Cpu className="size-5" />
     </div>
     <div>
```

Main application/interface
App.jsx

```
        <h1 className="text-lg font-semibold">CPU Scheduling Simulator</h1>
        <p className="text-sm text-muted-foreground">
         Non-preemptive FCFS &amp;amp; SJF · waiting and turnaround time
        </p>
       </div>
      </div>
      <ThemeToggle />
    </header>

    <main className="mx-auto grid max-w-5xl gap-6 px-6 pb-16 lg:grid-cols-[380px_1fr]">
     <div className="space-y-6">
      <Card>
       <CardHeader className="flex flex-row items-center justify-between gap-4 pb-6">
        <CardTitle>1. Number of processes</CardTitle>
        <Select value={countInput} onValueChange={handleSetCount}>
         <SelectTrigger className="w-20">
          <SelectValue placeholder="—" />
         </SelectTrigger>
         <SelectContent>
          {Array.from({ length: 8 }, (_, i) => i + 3).map((n) => (
           <SelectItem key={n} value={String(n)}>
            {n}
           </SelectItem>
          ))}
         </SelectContent>
        </Select>
```

Main application/interface
App.jsx

```
        </CardHeader>
      </Card>

      {rows.length > 0 && (
       <Card>
        <CardHeader>
         <CardTitle>2. Process details</CardTitle>
         <CardDescription>
          Enter each process ID, arrival time, and burst time.
         </CardDescription>
        </CardHeader>
        <CardContent className="space-y-4">
         {rows.map((row, i) => (
          <div key={i} className="grid grid-cols-3 gap-2">
           <div className="space-y-1">
            {i === 0 && <Label className="text-xs text-muted-foreground">Process
ID</Label>}
            <Input
             value={row.id}
             onChange={(e) => updateRow(i, "id", e.target.value)}
             placeholder={`P${i + 1}`}
            />
           </div>
           <div className="space-y-1">
            {i === 0 && <Label className="text-xs text-muted-foreground">Arrival</Label>}
            <Input
```

Main application/interface
App.jsx

```
                type="text"
                inputMode="numeric"
                pattern="[0-9]*"
                value={row.arrival}
                onChange={(e) =>
                  updateRow(i, "arrival", e.target.value.replace(/\D/g, ""))
                }
                placeholder="0"
              />
            </div>
            <div className="space-y-1">
              {i === 0 && <Label className="text-xs text-muted-foreground">Burst</Label>}
              <Input
                type="text"
                inputMode="numeric"
                pattern="[0-9]*"
                value={row.burst}
                onChange={(e) =>
                  updateRow(i, "burst", e.target.value.replace(/\D/g, ""))
                }
                placeholder="1"
              />
            </div>
          </div>
        ))}
      </CardContent>
```

Main application/interface
App.jsx

```
      </Card>

    )}


    {rows.length > 0 && (

     <Card>

      <CardHeader>

       <CardTitle>3. Algorithm</CardTitle>

       <CardDescription>Pick which scheduling algorithm to run.</CardDescription>

      </CardHeader>

      <CardContent className="space-y-4">

       <Select value={algorithm} onValueChange={setAlgorithm}>

        <SelectTrigger className="w-full">

         <SelectValue />

        </SelectTrigger>

        <SelectContent>

         <SelectItem value="fcfs">{ALGORITHMS.fcfs}</SelectItem>

         <SelectItem value="sjf">{ALGORITHMS.sjf}</SelectItem>

        </SelectContent>

       </Select>


       <div className="flex gap-2">

        <Button onClick={handleCompute} className="flex-1">

         <Play />

         Compute

        </Button>

        <Button onClick={handleReset} variant="outline" size="icon" title="Reset">
```

Main application/interface
App.jsx

```
              <RotateCcw />
            </Button>
          </div>
        </CardContent>
      </Card>
    )}

    {errors.length > 0 && (
      <Alert variant="destructive">
        <AlertCircle />
        <AlertTitle>Please fix the following</AlertTitle>
        <AlertDescription>
          <ul>
            {errors.map((err, i) => (
              <li key={i}>{err}</li>
            ))}
          </ul>
        </AlertDescription>
      </Alert>
    )}
  </div>

  <div className="space-y-6">
    {!result ? (
      <Card className="flex min-h-80 items-center justify-center border-dashed">
        <CardContent className="pt-6 text-center text-sm text-muted-foreground">
```

Main application/interface
App.jsx

```
          Set your processes and click Compute to see the schedule,
          waiting time, and turnaround time here.
        </CardContent>
      </Card>
    ) : (
      <>
        <div className="grid grid-cols-2 gap-4">
          <Card>
            <CardHeader>
              <CardDescription>Average waiting time</CardDescription>
              <CardTitle className="text-3xl">
                {result.averageWaiting.toFixed(2)}
              </CardTitle>
            </CardHeader>
          </Card>
          <Card>
            <CardHeader>
              <CardDescription>Average turn-around time</CardDescription>
              <CardTitle className="text-3xl">
                {result.averageTurnaround.toFixed(2)}
              </CardTitle>
            </CardHeader>
          </Card>
        </div>

        <Card>
```

Main application/interface
App.jsx

```
          <CardHeader>
           <CardTitle>Execution timeline</CardTitle>
           <CardDescription>{ALGORITHMS[algorithm]}</CardDescription>
          </CardHeader>
          <CardContent>
           <GanttChart schedule={result.schedule} />
          </CardContent>
         </Card>


         <Card>
          <CardHeader>
           <CardTitle>Process results</CardTitle>
          </CardHeader>
          <CardContent>
           <Table>
            <TableHeader>
             <TableRow>
              <TableHead>Process</TableHead>
              <TableHead>Arrival</TableHead>
              <TableHead>Burst</TableHead>
              <TableHead>Completion</TableHead>
              <TableHead>Turnaround</TableHead>
              <TableHead>Waiting</TableHead>
             </TableRow>
            </TableHeader>
            <TableBody>
```

Main application/interface
App.jsx

```
                {result.rows.map((row) => (
                  <TableRow key={row.id}>
                    <TableCell className="font-medium">{row.id}</TableCell>
                    <TableCell>{row.arrival}</TableCell>
                    <TableCell>{row.burst}</TableCell>
                    <TableCell>{row.finish}</TableCell>
                    <TableCell>{row.turnaround}</TableCell>
                    <TableCell>{row.waiting}</TableCell>
                  </TableRow>
                ))}
              </TableBody>
              <TableFooter>
                <TableRow>
                  <TableCell colSpan={4}>Average</TableCell>
                  <TableCell>{result.averageTurnaround.toFixed(2)}</TableCell>
                  <TableCell>{result.averageWaiting.toFixed(2)}</TableCell>
                </TableRow>
              </TableFooter>
            </Table>
          </CardContent>
        </Card>
      </>
    )}
  </div>
 </main>
</div>
```

Main application/interface
App.jsx

```
  )

}
```